



D2.1

System specification

Revision v1.0

Work package	WP2
Task	T2.1
Due date	31-03-2027
Submission date	31-03-2027
Deliverable lead	PCSS
Version	V1.0
Authors	Tomasz Hoffmann (PCSS), Matej Durco (OEAW), Jeroen De Dauw (PWIKI), Andraž Repar (JSI), Lozana Rossenova (TIB)
Reviewers	George Bruseker (Takin), Antoine Isaac (EF), Jouni Tuominen (UH), Vera Maria Charvát (OEAW), Julia Alili (TIB)

Abstract

Definition of the system requirements, as well as the overall architecture and technical specification of the proposed solution to ensure ECHOLOT's full integration with ECCCH, DS4CH and other platforms.

Keywords

System specification; Software architecture; Requirements engineering; Interoperability;

Version	Date	Description of change	Contributor(s)
V1.0	30-06-2026	1 st version of deliverable	Tomasz Hoffmann (PCSS), Matej Durco (OEAW), Jeroen De Dauw (PWIKI), Andraž Repar (JSI), Lozana Rossenova (TIB)

Disclaimer

The information, documentation and figures available in this deliverable are provided by the ECHOLOT project's consortium under EC grant agreement **101233096** and do not necessarily reflect the views of the European Commission. The European Commission is not liable for any use that may be made of the information contained herein.

Copyright notice

This report is licensed under a Creative Commons Attribution 4.0 International Licence (CC BY 4.0). The full licence text is available at <https://creativecommons.org/licenses/by/4.0/>

Nature of the deliverable*: R**Dissemination level**

PU	Public, fully open. e.g., website	☒
CL	Classified information as referred to in Commission Decision 2001/844/EC	☐
CO	Confidential to ECHOLOT project and Commission Services	☐

*** Deliverable types:**

R: document, report (excluding periodic and final reports).

DEM: demonstrator, pilot, prototype, plan designs.

DEC: websites, patent filings, press and media actions, videos, etc.

OTHER: software, technical diagrams, etc.

Table of contents

1 Executive Summary	7
2 Introduction and Methodology	8
2.1 Human-Centred Design and Development Methodology	8
Figure 1. ECHOLOT's Double Diamond and C4 Model	9
2.2 C4 Model for Representation	9
3 Requirements Engineering	10
3.1 User Personas and Overall User Goals	11
3.2 User Journey Maps	12
3.3. Living Document: Functional and Non-functional Requirements	12
4 System Context (Level C1)	13
4.1 External Actors	13
4.1.1 System Users	13
4.1.2 Software Developers	14
4.1.3. System Re-users	14
4.2 External Systems	15
Figure 2. C1 – System Context Diagram	16
4.3 Data Model	16
4.4 Interoperability and Integration	17
4.4.1 Interoperability with Source Systems	18
4.4.2 Interoperability with Europeana & DS4CH	20
4.4.3 Interoperability with Wikidata	21
4.4.4 Interoperability with ECCCH	22
4.4.5 Integration with ECCCH	22
Table 1. Levels of integration for applications introduced in ECHOES (D3.1, p. 26) and foreseen implementation in ECHOLOT	23
5 Containers (Level C2) & Components (Level C3)	23
Figure 3. C2 – ECHOLOT Core Container and its main components	24
Figure 4. C2 – ECHOLOT Enrich Container and its main components	25
5.1 ECHOLOT Core	25
5.1.1. Backend	25
5.1.2 Frontend	26

5.1.3 Data Tier: Relational DB	26
5.1.4 Data Tier: Triple Store	26
5.1.5 Admin Dashboard	26
5.2 ECHOLOT Enrich	27
5.2.1 Workflow Engine	27
5.2.2 Workflow Engine Admin Interface	28
5.2.3 Import/Export & Transformation	28
Figure 5. C3 – Import and Transformation component interactions	29
Figure 6. C3 – Export and Transformation component interactions	29
5.2.4 Reconciliation	30
Figure 7. Reconciliation workflow prototype developed for testing Basque heritage data use cases	31
5.2.5 Enrichment	31
6 Development Timeline	32
7 References	33

Abbreviations

CH	Cultural Heritage
CHPRs	Cultural Heritage Practitioners and Researchers
DARIAH	Digital Research Infrastructure for the Arts and Humanities
DMP	Data Management Plan
DS4CH	Data Space for Cultural Heritage
EC	European Commission
ECCCH	European Collaborative Cloud for Cultural Heritage
ECHOES	European Cloud for Heritage Open Science
ECHOLOT	European Cultural Heritage Optimised Linked Open Tools
EF	Stichting Europeana
EITF	ECHOES Integration Task Force
FAIR	FAIR Data Principles: Findable, Accessible, Interoperable, Reusable
GDPR	EU data protection regulations
GLAM	Galleries, Libraries, Archives and Museums
GWDG	Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen
JSI	Institut Jozef Stefan
LLM	Large Language Model
PMO	Project Management Office
PSNC	Instytut Chemii Bioorganicznej Polskiej Akademii Nauk
PWIKI	Professional Wiki GmbH
TIB	Leibniz Information Centre for Science and Technology
UPV/EHU	Universidad del Pais Vasco/ Euskal Herriko Unibertsitatea
WP	Work Package

1 Executive Summary

This document outlines the system architecture and design principles of the European Cultural Heritage Optimised Linked Open Tools (ECHOLOT) system. Integrated as a service within the European Collaborative Cloud for Cultural Heritage (ECCCH), ECHOLOT makes the creation, reuse, and dissemination of high-quality, semantically rich, and interoperable cultural heritage data accessible to scholars and institutions of any size. The new end-to-end system relies on knowledge graphs and open-source software (notably MediaWiki suite) for the application of the FAIR data principles and further integration with Europeana, DS4CH, EOSC, and global knowledge commons of the Wikimedia platforms (Wikidata, Wikimedia Commons, etc).

Using a combination of AI-, knowledge-, and rule-based approaches, ECHOLOT delivers hybrid curation workflows that efficiently combine automated processing and human input to perform multilingual and multimedia annotation, description generation, and entity recognition. The service features user-friendly interfaces and modules for the detailed tracking and embedding of provenance and rights metadata.

The architectural core of ECHOLOT is grounded in a "human-in-the-loop" philosophy. By leveraging containerized AI microservices for automated enrichment while maintaining expert-led validation, the system ensures that the resulting Linked Open Data (LOD) is both scientifically reliable and interoperable. Acting as a data exchange hub, ECHOLOT further enables the bi-directional flow of data ("roundtripping"), allowing enriched metadata to return to the source GLAM (Gallery, Library, Archive, Museum) partners, while simultaneously populating the ECCCH "Digital Commons" and global open knowledge platforms.

2 Introduction and Methodology

The structure of this document is based on design principles and results of the **human-centred design methodology** used in ECHOLOT's overall project development, as well as the [C4 model](#), a developer-friendly approach to visualising software architecture.

2.1 Human-Centred Design and Development Methodology

Following best practices in agile software development and user-centered design, ECHOLOT's system is developed iteratively through five phases using the [Double Diamond Model](#), developed by Design Council UK and employed worldwide across public institutions, universities and large technology and product design companies, including Meta, Braun, and IDEO.

The Double Diamond is an effective model for the iterative development of innovative solutions to challenging problems. The first diamond helps to understand the problem – through **Discovery** and **Definition**.

The *Discovery* phase is instrumental in identifying the ecosystem of infrastructures and applications within which ECHOLOT aspires to operate (see 4 *System Context - Level C1*). This phase starts with ECCCH and extends to related projects funded under the HERITAGE-ECCCH calls, as well as the broader field of platforms and communities relevant to Cultural Heritage Professionals and Researchers (CHPRs), with a primary and exemplary focus on Europeana and Wikimedia initiatives. The requirements for the development of the ECHOLOT system and the desired integration mechanisms are gathered through a range of community engagement activities (see 3 *Requirements Engineering*).

The C4 model used for the representation of the System Architecture sits at the intersection of the two diamonds and provides clear and easy-to-understand translation of user requirements into developer-friendly software architecture visualisations.

The second diamond works through the system's **Implementation** and **Evaluation**. The process is highly iterative which means individual phases partly overlap to allow a feedback loop between them. For example, the *Definition* phase will start as soon as initial requirements have been gathered, so as to allow refinement of the *Discovery* process with preliminary deliberations on the *System architecture* and *Interoperability model*.

Our methodology adds an additional phase dedicated to the development of ECHOLOT's exploitation and long-term sustainability plan, which is fully integrated in the

decision-making processes connected to the design of the system architecture, rather than being an after-thought. Sustainability and system governance models are formulated in a separate deliverable: D7.3 Exploitation and sustainability model.

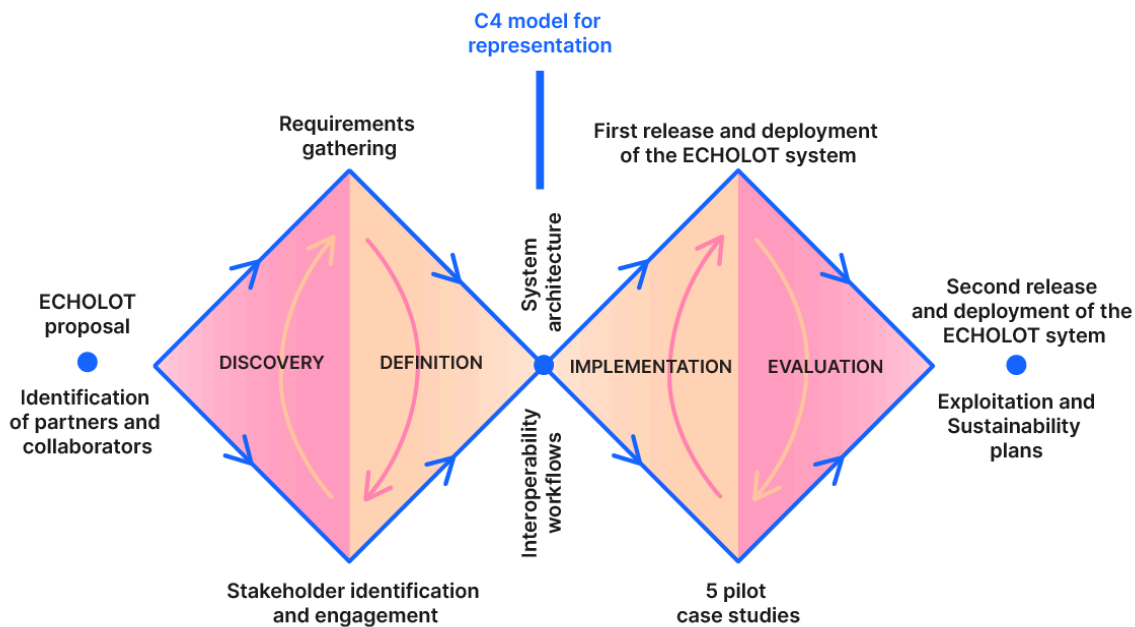


Figure 1. ECHOLOT's Double Diamond and C4 Model

2.2 C4 Model for Representation

The [C4 model](#) (Brown, 2026) provides a hierarchical way to visualize software architecture through four levels of abstraction, each serving a different audience and purpose.

Context (Level 1) shows the system from the highest vantage point. It places the software being built in relation to its users and external dependencies — other systems, third-party services, and people. The goal is to answer *what this system does and who uses it?* without any technical detail.

Containers (Level 2) zooms into the system to reveal its high-level technical building blocks: web applications, mobile apps, databases, microservices, and so on. A "container" here means a separately deployable or runnable unit, not a Docker container specifically. This level clarifies how responsibilities are distributed and how containers communicate.

Components (Level 3) drill into a single container to show its internal structure — the major logical groupings of code such as controllers, services, and repositories. This level is most useful for developers working within that specific container and helps map architecture to the codebase.

Code (Level 4) is the optional, lowest level, showing how a component is implemented using UML class diagrams or similar notations. Because it becomes outdated quickly, it is rarely recommended unless a particularly complex area warrants it. For this reason, we also don't describe Level 4 in this deliverable, but rather in technical documentation accompanying the developed software components.

How we use these levels is the key strength of the C4 model: rather than producing one overwhelming diagram, C4 affords tailored views for tailored audiences. Each diagram uses a consistent, simple notation — boxes and arrows with explicit labels — making it accessible without requiring UML expertise. The levels nest cleanly inside one another, so together they tell a coherent, drillable story of the entire system.

3 Requirements Engineering

The requirements engineering process in ECHOLOT, resulting from the Discovery phase, involved close work with the stakeholders directly involved in ECHOLOT's five pilot case studies. Representatives of over 20 cultural heritage and research institutions participated in filling questionnaires, data surveys and a two-day in-person workshop in Bilbao in March 2026, hosted by ECHOLOT partners UPV/EHU. Design methodology tools such as user personas,¹ user stories,² and journey maps³ helped our team to define an initial set of system requirements based on real-world user needs, goals and pain-points.

¹ A user persona is a realistic, research-based profile of a target user. It summarizes who the user is, what they're trying to achieve, their needs, behaviors, and pain points—it is a tool we use to design with a clear, shared understanding of who we're building ECHOLOT for.

² A user story is a short, simple description of a feature or need from the user's perspective. We also used user scenarios to combine stories into a brief narrative that describes how a specific user (often represented by a persona) interacts with a product to achieve a goal in a particular context—it helped us understand real-world usage and design for complete workflows rather than isolated features.

³ A user journey map is a structured overview of the end-to-end experience a user has when interacting with a service over time. It maps the phases, steps and concrete touchpoints users have with the service. We used this tool to map out an ideal user journey for four specific end-to-end workflows in ECHOLOT.

3.1 User Personas and Overall User Goals

In a co-design session, together with ECHOLOT case study partners, we identified a minimal set of user personas, which guide the development needs of the core system. These include the Museum Curator, the Digital Collections Manager, the Digital Humanities Researcher and the Wikimedian-in-Residence.⁴

Their overall goals are summarised below:

- Enabling institutional networking and cross-platform collaboration across departments, institutions, and user communities;
- Facilitating sharing, discovery, and exploration of specialized domain knowledge, research outputs, and cross-institutional collections;
- Supporting semantic interoperability, bi-directional data exchange (including roundtripping) between local systems, external repositories, infrastructures and standards;
- Supporting the transformation of research outputs into structured datasets through robust data modelling, ontology development, and semantic enrichment using external knowledge organization systems, controlled vocabularies, taxonomies, thesauri, and ontologies (e.g., SKOS, CIDOC CRM, Getty AAT, ECHOLOT-supported resources);
- Enabling the creation, integration, and management of heterogeneous cultural heritage collections and archival materials, including digitised cultural heritage objects and newly acquired records;
- Supporting complex relationship modelling, linking, contextualisation, and narrative enrichment across heterogeneous datasets and objects, including the identification of hidden relationships and gaps in data;
- Supporting metadata and rights management workflows, including copyright research, documentation, semantic rights metadata, and triaging of copyright levels.

These concrete, real-world goals set up a goals-oriented framework for the system context (Level C1) of ECHOLOT. And while some of the goals expand beyond the scope and strategic objectives of ECHOLOT as an individual system, they do align with the ECCCH ecosystem as a whole.

⁴ Read the full profiles on the ECHOLOT website: <https://echolot-eccch.eu/user-centered-design/>

3.2 User Journey Maps

After the definition of user personas with concrete goals, the team worked with our case study partners to define detailed user journey maps for each of the core workflows facilitated by the ECHOLOT system:

- [Import and validation](#)
- [Reconciliation and Enrichment](#)
- [Media annotation](#)
- [Export and roundtripping](#)

The journey maps serve as check-points between project stakeholders and developers – ensuring the functional requirements and the planned system containers (Level C2) and components (Level C3) align with the expected behaviour of the system during each step of the data processing workflows.

3.3. Living Document: Functional and Non-functional Requirements

A living document of the requirements is structured via issues and sub-issues in a dedicated [GitHub repository](#) which can be versioned, iteratively updated and directly linked to technical development issues in the repositories hosting the code of the core ECHOLOT system and AI-supported micro-services. Below is the list of the main categories of functional (FR) and non-functional (NFR) requirements as well as constraints organised as main issues in GitHub:

1. FR: ECHOLOT system | Interoperability & standards
2. FR: ECHOLOT system | User experience and interaction
3. FR: ECHOLOT system | Provenance & (Copy)rights management
4. FR: ECHOLOT system | AI-support
5. FR: Import & Export / Transformation pipeline | Data ingestion & integration
6. FR: Import & Export / Transformation pipeline | Data exchange, publishing, synchronization & roundtripping
7. FR: Reconciliation workflows
8. FR: Enrichment workflows
9. FR: Advanced image recognition & annotation workflows
10. FR: Automated quality checks & data validation workflows

11. NFR: ECHOLOT system
12. Constraints (setting a context and/or limitations)

4 System Context (Level C1)

ECHOLOT occupies a strategic position as the interoperability hub between local heritage repositories, the broader European research infrastructure and the open knowledge commons ecosystem of Wikimedia platforms. As such, the system context consists of a mix of diverse actors and external data and software architecture dependencies.

4.1 External Actors

As an ECHOES sister project, ECHOLOT engages eight strategic target groups (TG) which can all be considered external actors in the system's context.

4.1.1 System Users

TG1 & TG2: CH professionals at small CHIs or large national institutions (*corresponding to The Museum Curator persona (see 3.1)*)

ECHOLOT will help this group to perform a wide range of tasks with greater efficiency at a lower learning barrier, including but not limited to preparing data in a structured format for publication to national or thematic catalogues; disseminating data to multiple catalogues and knowledge platforms (including the Wikimedia ecosystem) and making it accessible to broader audiences; applying accurate license and rights metadata. Publication to data aggregators will be easier and not duplicating parallel efforts to reach the general public via Wikimedia channels.

TG3: Data stewards at research institutes or academic libraries (*corresponding to The Digital Collections Manager persona (see 3.1)*)

ECHOLOT will ensure data stewards can apply FAIR principles to all datasets under their care, through automating enrichment, entity linking to global authorities, generation of persistent identifiers and more. Interoperability with EOSC, ECCCH and DS4CH will help data stewards ensure they meet institutional as well as national mandates and make their data available for research.

TG4: Digital Humanities (DH) researchers (*and corresponding persona (see 3.1)*)

ECHOLOT will help DH researchers to get access to and work with CH data more easily, keep their generated data FAIR, and retain detailed chain-of-production provenance data, ensuring the knowledge generation process is reproducible and reliable, preserving the IP of the original research contribution.

4.1.2 Software Developers

TG5: Research software engineers at CHIs or research institutes

ECHOLOT makes the administration and reuse of its components and APIs easy and accessible to other software engineers. Furthermore, ECHOLOT's system is available as open source software that can be readily deployed locally for experimentation, and reuse in bespoke systems.

TG6: Open source MediaWiki developers

MediaWiki developers are an active international community contributing to the continuous development of the open source MediaWiki software. ECHOLOT will make available multiple new open source modules and extensions, introducing improved interfaces and automation into the existing software suite, which can be reused, improved and developed further, welcoming active contributions from this target group beyond the project's end.

4.1.3. System Re-users

TG7: Citizens (*a core concern for the Wikimedian-in-Residence persona (see 3.1)*)

ECHOLOT offers citizens participating in grassroots knowledge initiatives to become part of the interconnected knowledge ecosystem of linked open data, made possible through the integration of multiple knowledge platforms including Wikimedia, DS4CH, Europeana, ECCCH, EOSC. Citizens contributing new information to a common public body of knowledge will benefit from data enriched by CHPRs, while receiving recognition for their own contributions to enrich institutional datasets in return.

TG8: Cultural & creative industries

ECHOLOT renders reuse of CH data more easily accessible to the cultural and creative industries, providing clear information on conditions of reuse and APIs with effective means for automation, stimulating the development of new products and services.

4.2 External Systems

ECHOLOT is embedded in a complex ecosystem comprised of many heterogeneous systems:

- **ECCCH (ECHOES):** The primary cloud infrastructure providing core services including AAI/SSO, the Knowledge Base (KB), the Integration Manager and more.
- **MediaWiki ecosystem:** The technical core of the ECHOLOT system is conceived as an extension to the MediaWiki software suite, correspondingly the enormous MediaWiki ecosystem (code, extensions, contributors) represents a critical context to ECHOLOT.
- Systems for management and publication of software artefacts, code, libraries, container images, ML models, primarily GitHub as the code repository system also used by ECHOES, or Hugging Face as the most widely used platform for sharing ML-related resources.
- **Source systems:** Local repositories and databases of CH institutions holding information about CH objects serving as primary data sources for ECHOLOT. (in many different formats and access pathways). These are represented chiefly through the CH partner institutions involved in the case studies (WP5). An initial survey among these institutions revealed a great heterogeneity in the systems and data models employed.
- **Target systems:** Bringing together information about CH objects from multiple sources. Primary target systems are:
 - DS4CH/Europeana Aggregators: Target endpoints for high-quality metadata publication via the common European data space for cultural heritage.
 - Public Knowledge Platforms (Wikimedia Ecosystem): Integration targets for open-data dissemination, specifically Wikidata and Wikimedia Commons.
- **Reference Resources:** Systems for management and publication of reference resources like controlled vocabularies, thesauri, gazetteers, knowledge bases (GND, Geonames, VIAF); **NB: Wikidata** has a special role both as a target system and as a reference resource.
- **Hosting infrastructure:** During the project ECHOLOT system will be deployed on the server infrastructure of selected project partners. Once ready, it should also be deployable as a vertical application on the ECCCH cloud infrastructure.

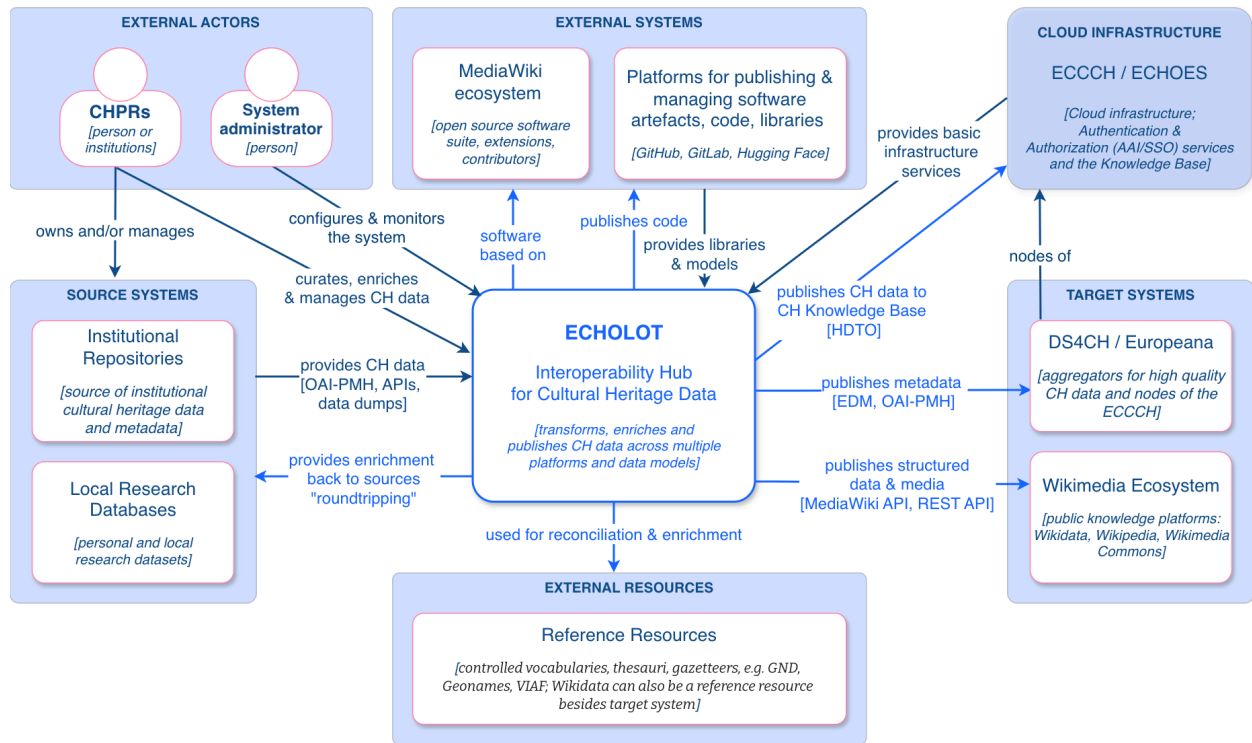


Figure 2. C1 – System Context Diagram

4.3 Data Model

As described above, ECHOLOT will operate in a complex ecosystem of heterogeneous data models, with its core mission to serve as an interoperability hub that transforms data between these models and enables exchange between systems. This requires the ECHOLOT system's internal data model to be flexible and expressive enough to capture heterogeneous information losslessly.

The internal data model is also influenced by the underlying system, the newly developed NeoWiki extension of the MediaWiki software suite (see 5.1). The system will provide a few generic modeling mechanisms that allow us to define concrete data shapes dynamically and represent them within the system. A key element of the new system is the so-called **Schemas**, which represent specific types of entities and their relations. The main modeling concepts of the NeoWiki system are specified in its [documentation](#).

Inheriting from the underlying MediaWiki system, the basic generic entity is a **Subject**, (corresponding to a Wikidata/Wikibase Item) with an id, type, label, and set of related data

statements. Each statement captures a specific aspect of the described entity and is a triple of *propertyName*, *propertyType*, and *Value*. (Example: Property Name "age" with Value 42 and Property Type number). Schemas define which properties can be used to describe specific entities (e.g. name, birth date, profession for a Person).

The ECHOLOT system will define standardized Schemas appropriate for cultural heritage data management activities (e.g.: authority entities for places, persons etc., digital objects, publications, bibliographic entries) that will be co-defined and developed with the user communities as represented by the case study stakeholders. In light of the core aim of standardization, the Schemas will be defined and organized based on international standard ontologies (e.g.: EDM, Wikidata, HDTO, CIDOC-CRM) according to the needs and requirements identified by the user communities.

Another crucial aspect of the ECHOLOT system will be **mappings** defining transformations between data models. These could take the form of simple cross-walk description, but the larger goal is to create full executable mapping instruction descriptions to support multi publication of data from one standard to another. The set of mappings required will be fully determined through the iterative discussion with use case partners in WP5, but the minimal set presently envisioned includes Wikidata and the Europeana data model, as well as the eventual HDTO model.

Provenance and rights metadata are modeled with Schemas, the same as everything else, not built into the generic modeling mechanism itself. The resulting records capture which sources, processes, and actors produced the data, and stay with it, readable by humans and machines.

4.4 Interoperability and Integration

For the terms interoperability and integration, ECHOLOT relies on the definitions used in the ECHOES project, especially in deliverables:

- D3.1 ECHOES Integration Strategy for datasets, tools and workflows with potential for reuse in ECHOES: <https://doi.org/10.5281/zenodo.17751335>
- D3.2 ECHOES Integration Roadmap: <https://doi.org/10.5281/zenodo.17751670>
- D6.2 ECHOES Interoperability Requirements and Guidelines: <https://doi.org/10.5281/zenodo.18656368>

[D6.2 Interoperability Requirements and guidelines](#) states (p.19):

Interoperability vs. Integration

Although often used interchangeably, interoperability and integration describe distinct concepts.

Interoperability refers to the ability of systems to communicate and use exchanged information meaningfully, based on shared standards and agreed conventions, without requiring system-specific adaptations.

Integration, by contrast, refers to the architectural effort required to make systems work together. It typically involves tightly coupled solutions, custom connectors, middleware, and transformation layers that align heterogeneous systems into a single operational setup.

Interoperability is a prerequisite of Integration.

The ECCCH represents the primary infrastructural context for ECHOLOT (as required by the [corresponding project call](#)), which implies that the outputs of ECHOLOT must meet the interoperability requirements set by ECHOES as prerequisite for their integration with ECCCH.

At the same time ECHOLOT's positioning as an interoperability hub facilitating information exchange between numerous heterogeneous systems (as laid out in 4.2), can be defined to a large extent in terms of interoperability with these systems.

The following sub-sections elaborate on these aspects divided by the different external systems involved.

4.4.1 Interoperability with Source Systems

Consolidating and homogenising heterogeneous information about CH objects held in custom local systems is one of the central promises of ECHOLOT (implemented mainly in the Import module (T4.1)).

The initial survey among data providers involved in the case studies (WP5) revealed a large variety of employed systems, offering different means to access the data, however there are also certain overlaps.

Even though it will not be feasible to support any source system, ECHOLOT will aim for a flexible import mechanism that can handle the most common protocols and data models out of the box and will be easily extensible to support new scenarios based on demand.

Extract from case studies survey (February 2026):

How do you expose your data (e.g., via an API and / or IIIF, bulk-download, etc.)?	What software product(s) do you use for managing and publishing your data?
Public project website (in Croatian) and in simplified tabular form in a printed monograph (Croatian and English editions). The dataset is not yet available via API, IIIF, or bulk download, but these are planned for future phases of the project if suitable.	The data are currently managed in Excel and maintained through our own research and editorial workflow. For publication, the table has been transferred to a CMS for the project website, and a simplified version is also available in printed form. No dedicated collection management system or repository software is used at this stage.
REST API, IIIF image API, IIIF Presentation API, IIIF Content Search API, Elasticsearch	Filemaker, Apache Kafka, MongoDB, Own development based on Spring Framework
RDF/LOD/SPARQL pointing to access links	Wikibase, Snowman
Wiki - webinterface	Wikibase Cloud
API is planned	Individual scripts for harmonisation and processing. For presentation: customised version of VuFind discovery system.
API	Inhouse built, web-based application.
At the moment it is not exposed yet. We are working towards API's, IIIF and Bulk downloads. We are still debating whether or not to include SPARQL endpoints. The project we are working on needs to follow the 5 checkpoints of the NDE (Network Digital Heritage) which are: the use of persistent identifiers: a permanent web address for each collection object; the use of common terms via the Term Network; the publication of collection data as linked (open) data; the publication of images as IIIF; the registration of your collection data in the Dataset Register.	We are using: mysql, mongodb, php, NODJS. The software is developed specifically for our collection.
We expose them to human users only. We can export data in table formats from our databases. From the repository, items are available through OAI-PMH (items already sent to Europeana only).	Library software, VUFIND for the public catalogue discovery layer, DSPACE for the repository.

The catalogue has an export function (Z39.50); The repository has an API (OAI-PMH)	Custom XML database
Expose collection data primarily via APIs. Aggregated metadata is made available through datahub via an OAI-PMH endpoint (e.g. LIDO as metadata format). Digital representations are accessed via imagehub through stable link.	The Museum System for managing our collection metadata and a Resourcespace for managing digital representations. For publication and data exchange, a dedicated datahub infrastructure that harvests and aggregates metadata from the source systems and exposes it via API (OAI-PMH compliant).
At the moment, only our metadata database is publicly available online; We do not (yet) use API's or IIIF to expose our data.	We use our own custom database, and ResourceSpace for our DAMs.
SQL database: API. Wikibase: several options (MediaWiki action API, SPARQL)	We have our own database management UI for cataloguing tasks. The Wikibase is a wikibase.cloud instance.
No bulk exposition at the moment, bulk download necessary.	MuseumPlus and eMuseumPlus / Qbanks Mediaportal
OAI-PMH, IIIF	Axiell Collections (soon Museumplus)
API and IIIF	Axiell Collections, ResourceSpace, VKC eco system
We don't have our own API. Images from Resourcespace are available via IIIF.	We have a Drupal-based website. There is a cron job that connects via API to Axiell and Resourcespace and then creates custom 'artwork' nodes in Drupal.
Digital Museum API, Bulk download	Primus, X-ref
No bulk exposition at the moment (bulk download necessary).	BeeCollect; NetX

4.4.2 Interoperability with Europeana & DS4CH

Europeana and DS4CH are considered target systems for ECHOLOT, i.e. ECHOLOT should be able to provide data to these systems. This is the responsibility of the Export module (T4.1), which will allow the transformation of data processed via ECHOLOT's core system to a range of target systems.

More specifically, Europeana (and the data space built based on its infrastructure) relies primarily on aggregation of metadata using the [OAI-PMH](#) protocol and the [Europeana Data Model \(EDM\)](#), which means ECHOLOT will have to ensure interoperability with these standards as a minimum viable product. Alternatives can be envisioned as the DS4CH develops new ways for ingesting datasets. These include Linked Data harvesting from SPARQL endpoints and upload of datasets that do not follow the current approach of item-level descriptions in EDM - focusing rather on dataset-level descriptions expressed using [Datasheets for Digital Cultural Heritage Datasets](#) (soon to be implemented as a profile of the [DCAT-AP](#) specification used in many European projects and data spaces).

As more details become available regarding the evolution of the data space, ECHOLOT will respond to those requirements as well. Further requirements may also emerge at the moment where data aggregation paths will be determined for the various ECHOLOT partners willing to contribute data to the DS4CH, as part of the active phase of WP5. Indeed, the DS4CH does not in general directly handle ingestion with its many data partners: instead, data aggregators work with cultural institutions to gather data and channel it into the data space. These aggregators can have their own (meta)data requirements, for example the use of a specific data model – or a specific application profile of EDM – which ECHOLOT may then want to provide support for.

Finally, we are aware that technical interoperability with DS4CH formally includes more criteria than these of metadata model and protocols alone: in particular, the Europeana Licensing Framework governs the licensing conditions that apply to metadata and content being submitted to the DS4CH, and the Europeana Publishing Framework sets up expected levels of quality for the data (and how these shall be measured). However, at this stage it is unclear whether these frameworks have a direct impact on how the ECHOLOT system shall be designed and implemented, because they regard aspects of data contribution that are either: (i) external to its currently apparent scope – such as quality assessments, which are by default carried out by DS4CH's own ingestion components; (ii) optional – like the provision of content using the International Image Interoperability Framework, IIIF; or (iii) will be already considered as part of ECCCH interoperability and integration recommendations (IIIF, again).

4.4.3 Interoperability with Wikidata

Interoperability with Wikidata requires reconciliation to Wikidata entities (via the reconciliation service and/or the Wikidata vector database and Wikidata MCP) and upload to Wikidata, via the MediaWiki API or REST API, following its RDF-based data model. Wikidata's

ontology includes a large array of properties, which do not have formally defined domain and range constraints, while classes are also not organised in a formal way (there is no Tbox / Abox division in its knowledge base), leading to a range of challenges for interoperability. However the dedicated tasks (T2.3) and co-design activities (WP5) of ECHOLOT's programme have accounted for these challenges and will rely on the domain expertise and expertise from the Wikimedia communities which are part of the ECHOLOT consortium and third-party stakeholders to create detailed mappings from source data standards to appropriate, and community-accepted Wikidata ontological concepts and entity schemas.

4.4.4 Interoperability with ECCCH

This interoperability effort is guided by ECHOES' deliverable D6.2 <https://doi.org/10.5281/zenodo.18656368>, which defines technical interoperability requirements (chapter 3) that need to be met by all resources to be onboarded (i.e. integrated) into the CH Cloud. Different requirements apply to different resource types (Datasets, Applications, APIs & Web Services, Semantic Artefacts, Workflows). The requirements are complemented with guidelines that explain how providers can implement the requirements in practice. The deliverable is accompanied by living technical documentation: [ECHOES Engineering Handbook](#).

Requirements are grouped into three interoperability levels – Basic, Intermediate, Advanced – corresponding to three integration levels (see next section).

ECHOLOT's outputs will be checked and validated against these requirements as precondition to integration to ECCCH.

4.4.5 Integration with ECCCH

In line with the definition introduced at the beginning of the section 4.4, interoperability is precondition for integration, integration meaning the effort to make systems work together.

ECHOES has outlined a strategy, including rules and procedures for integration of resources ([D3.1 Integration Strategy](#), [D3.2 Integration Roadmap](#)), especially from sister projects like ECHOLOT. The process is coordinated by the ECHOES Integration Task Force (EITF), which interacts with all the associated projects and is primary contact also for ECHOLOT. The process foresees the identification of so-called *Integration Resources* and *Integration Units*, the latter defined as a pair consisting of a project resource and a cloud component it should integrate with. ECCCH will provide a set of cloud components which the ECHOLOT system can or must use (especially AAI and CH Knowledge Base). Elaborate specification of the cloud components will be provided in D6.3 to be released 2026-06.

For ECHOLOT the main resource will be the *ECHOLOT Suite* Application, which will be subject to multiple integrations (separate Integration Units, tentative):

- ECHOLOT Suite and AAI: allowing users to login using single-sign on via their home institution
- ECHOLOT Suite and CH Knowledge Base: reading and writing information about CH objects in HDTO
- ECHOLOT Suite and Cloud deployment: deploy ECHOLOT as an on-demand service.

Integration Level	Technical Perspective	Implementation in ECHOLOT
Level 1: Tactical (Basic Connectivity)	IaaS (Infrastructure as a Service): Users spin up a generic VM to run the software manually. The cloud acts merely as a host.	NeoWiki Suite deployable via dedicated VMs on the Cloud infrastructure;
Level 2: Strategic (Semantic & Technical Integration)	Platform as a Service (PaaS): Applications use Cloud APIs for Auth(AAI), Logging, and Storage. They are loosely coupled.	NeoWiki Suite allows single sign on via federated identity; interacts (read/write) with the Cloud Knowledge Graph;
Level 3: Transformational (Federated Ecosystem Participation)	SaaS / FaaS (serverless) Tools are containerised (Docker/Kubernetes). They are deployed as Microservices that scale automatically based on demand.	AI-supported data enrichment microservices running on the Cloud; publishing chain-of-production provenance metadata to the Cloud Knowledge Graph;

Table 1. Levels of integration for applications introduced in ECHOES (D3.1, p. 26) and foreseen implementation in ECHOLOT

5 Containers (Level C2) & Components (Level C3)

The ECHOLOT architecture is partitioned into two main containers:

- **ECHOLOT Core:** The primary data curation environment of the ECHOLOT system, built on a customized **MediaWiki software extension, NeoWiki**.

The NeoWiki backend manages and mediates access to data. Data is stored in a Relational DB and a triple store.

The NeoWiki frontend facilitates user interactions and various data curation and presentation tasks, including human-in-the-loop validation of AI-supported hybrid curation workflows.

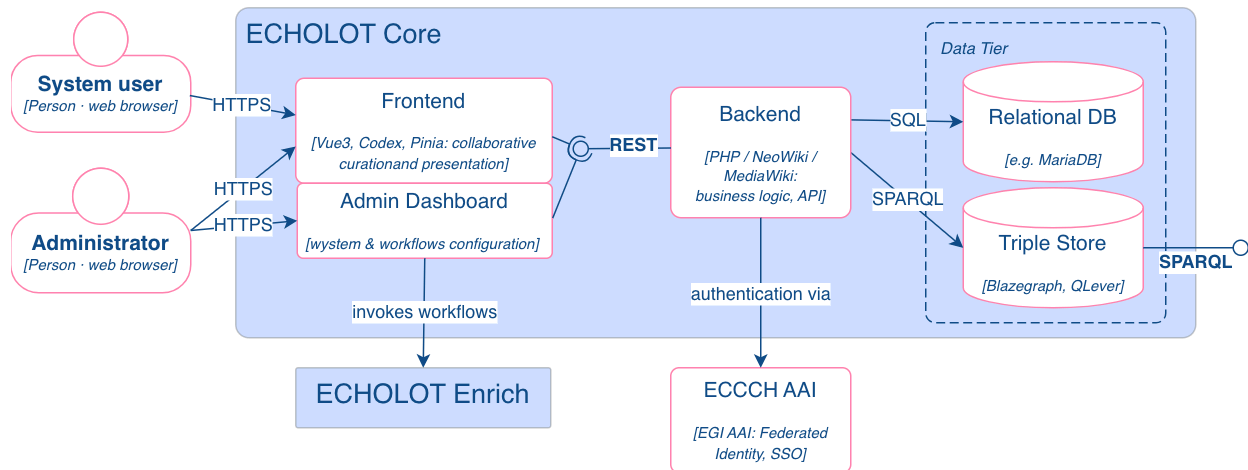


Figure 3. C2 – ECHOLOT Core Container and its main components

- **ECHOLOT Enrich:** This container encompasses all the components that perform automated operations on the data stored in ECHOLOT as part of the hybrid data curation workflows.

They are implemented as containerized microservices employing AI-based methods (like **Retrieval-Augmented Generation (RAG)** and **In-Context Learning (ICL)**, among others), alongside classic knowledge- and rule-based approaches. These services rely heavily on verified external reference resources to generate and/or enrich structured metadata in a reliable manner.

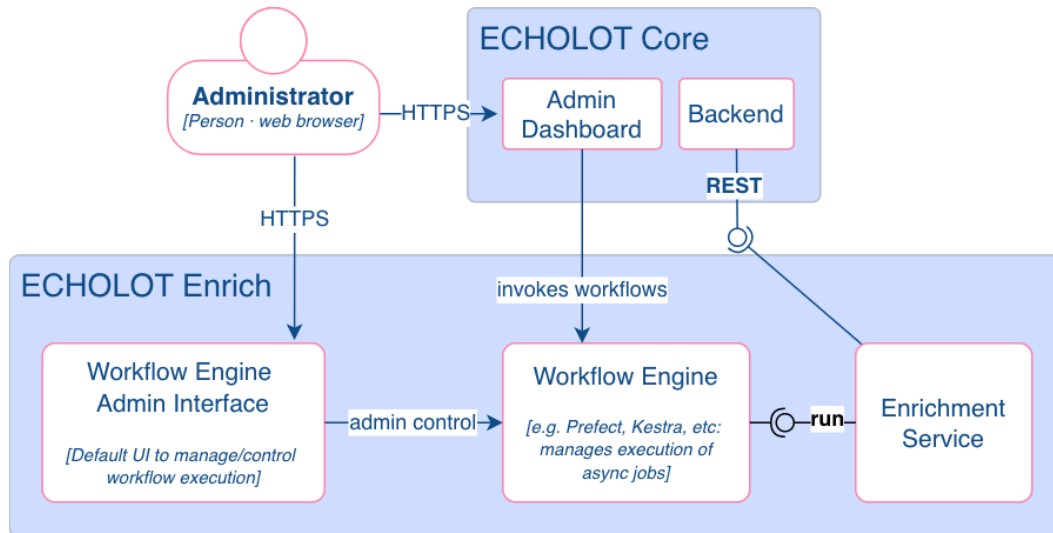


Figure 4. C2 – ECHOLOT Enrich Container and its main components

There is a clear separation of concerns between the containers and these will respectively communicate via well-defined APIs. However, from a user's perspective, containers shall be perceived as one coherent, integrated system. The most challenging aspect will be the implementation of the hybrid “human-in-the-loop” workflows, especially in terms of the user interaction challenges borne out of emerging agentic workflows which have to date not been extensively researched and tested by UX designers and researchers.

Deployment Layer: To ensure infrastructure-agnostic resilience, the system is delivered as Docker containers, de facto virtualisation technology, allowing to deploy the system easily on any host infrastructure. During the project's lifetime an initial set of five instances (one per each case study) will be deployed on the cloud infrastructures of selected project partners (PCSS, TIB). However, the standard setup will also allow local deployment for development environments, as well as deployment on the ECCCH cloud infrastructure.

5.1 ECHOLOT Core

5.1.1. Backend

The Backend is NeoWiki's server-side component, a MediaWiki extension written in PHP, and owns the domain logic for the data model described in 4.3. The design is domain-centric: the logic remains independent of MediaWiki and storage backends, so it can be tested on its own, and storage backends can be swapped. The Backend validates data

against its Schemas and persists it as the versioned source of truth, keeping the graph projection in sync on every change. All data access runs through it, so access rights are enforced in one place, including on query results. A REST API handles reads and writes at the subject, bulk, and statement levels, alongside a read-only SPARQL query endpoint over the triple store. Import, reconciliation, enrichment, and quality-check workflows plug in through its extension points.

5.1.2 Frontend

The Frontend is a browser application built with Vue 3, the Codex design system and Pinia. It provides the editors for schemas, subjects and layouts, and renders structured data on wiki pages as infoboxes and other views. The workflow review steps run here, including import preview, reconciliation matching, enrichment, and validation feedback. All data passes through the Backend's REST API, never directly to the databases. Codex keeps the interface accessible and consistent with other MediaWiki software and lets wiki skins control its appearance.

5.1.3 Data Tier: Relational DB

The Relational DB is the MediaWiki database, running on MySQL or MariaDB. It holds the source of truth as versioned JSON in dedicated revision slots, stored next to each page's wiki content. Every edit retains MediaWiki's full revision history, including author, timestamp and the change from the previous version. The database also stores standard wiki data such as pages, users and access rights. The Triple Store is a projection of this data and can be rebuilt from it.

5.1.4 Data Tier: Triple Store

The Triple Store holds the RDF projection of the data and answers SPARQL queries. NeoWiki targets any SPARQL 1.1 compliant store, such as QLever, chosen at deployment time. The Backend writes each page's triples into a separate named graph, which makes updates and deletions atomic and records which page each statement came from. The mapping produces NeoWiki's own predicates and can also align the data to standard ontologies such as CIDOC-CRM and EDM. A public read-only SPARQL endpoint and RDF dumps let other Linked Open Data systems query and federate with the wiki.

5.1.5 Admin Dashboard

User-facing component mainly to be used by system administrators that allows them to perform administrative tasks, configure the system, control the execution of automated

processing tasks. It interacts with the *Workflow Engine*, allows to parametrize, configure and start processing jobs, oversee their execution and inspect the results (or failures). It may be limited in functionality in comparison to the native Admin Interface of the Workflow Engine.

Additional [documentation](#) of the NeoWiki current development including a [Glossary](#) is available on the NeoWiki Github repository.

5.2 ECHOLOT Enrich

The **ECHOLOT Enrich** container encompasses all the components that perform automated operations on the data stored in ECHOLOT as part of hybrid data curation workflows, as well as to import data from and export into external systems (as introduced in 4.2).

Some common principles for these components:

- Expose a unified interface, via which they can be invoked by the workflow engine.
- Write the output (changes to existing items, or creation of new items in ECHOLOT) directly to the ECHOLOT Core using the main REST API (as new revisions of given records).
- Support test mode, where the results are not yet written to the system, but can be inspected. The underlying scenario is that when the system administrator sets up an enrichment task on a given dataset, they run it on a small sample to validate the configuration, and only once satisfied, launch the task as a background batch process on the whole dataset.
- Output/return a report log, providing meta information about the process. i.e. a summary of changes, listing of errors. It has to be easily possible to get from an administrative handle of a job to the affected records.
- Hybrid *human-in-the-loop* curation workflows – an iteration of the automated process suggests changes, which are then reviewed by a human; with the revisions serving as input for another iteration of the automated process.

5.2.1 Workflow Engine

The Workflow Engine is the component managing the automated, bulk data processing tasks. The system administrator interacts with it via the Admin Dashboard component (integrated in ECHOLOT core, including a limited set of functionalities and lower learning curve), or via the native Admin interface (including full range of functionalities, but also higher learning curve).

For this task, ECHOLOT will adopt one of numerous existing workflow and orchestration automation systems, such as Prefect, Kestra, Airflow, Galaxy, n8n. (cf. comprehensive [review of existing workflow/orchestration](#) tools), based on an evaluation of the candidate systems.

5.2.2 Workflow Engine Admin Interface

Even though the enrichment tasks are expected to run primarily as batch processes on (potentially large) datasets, they still require UI/frontend components for administering distinct functions:

- admin dashboard to manage, start and monitor batch processing jobs;
- UI to explore the results of the processing;
- UI for AI-assisted data curation tasks, that allows for “human in the loop” interventions;
- UI for interacting (chatting) with an Agent that suggests and performs operations on the data;

In order to avoid developing too many components from scratch, the default interface of the Workflow Engine (e.g. Prefect) will be implemented in the first version of the running ECHOLOT system (due M15). Following the WP5 testing and evaluation, a more comprehensive interface may be implemented directly in ECHOLOT Core during WP6 iterative development. It is worth noting that the AI-assisted curation is a completely novel approach that will require thorough rethinking of the user interactions and UI/UX design.

5.2.3 Import/Export & Transformation

As stated in 4.4, central to ECHOLOT’s mission is the role of an interoperability hub able to read data from and write to heterogeneous systems. This functionality is implemented in the components Import, Export and Transformation. Transformation implementing the translation of data between external models and data formats and the internal data representations (‘Schemas’, cf. 4.3 Data model) is used by both Import and Export components, which handle primarily the reading and writing operations from and to external systems.

Even though it is not feasible to support any source system, ECHOLOT will aim for a flexible import/export mechanism that will be able to handle the most common protocols and data models out-of-the-box and will be easily extensible to support new scenarios on demand. The initial set of protocols and data models will be based on the configurations found with the data providers of the case studies (WP5) as well as dictated by interoperability requirements of the primary target systems (ECCCH Knowledge Base, Wikidata, Europeana).

The system also needs to offer some representation of the source data, as a basis for an interactive import process, e.g. it could read column headers from a CSV-table and make them available as configuration parameters of the transformations. Ideally it could even suggest suitable mapping based on previous imports and AI assistance.

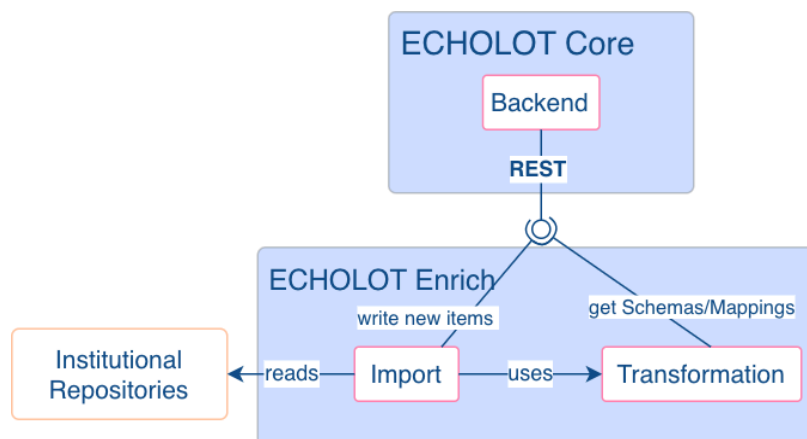


Figure 5. C3 – Import and Transformation component interactions

Export will be able to write data to external systems (push), again supporting a set of established APIs. Complementary, data will be exposed via APIs for other systems to consume (pull), however this will be covered by other components: Next to the main REST API provided by the core component and the SPARQL endpoint of the underlying triple store, there will be also an OAI-PMH endpoint, probably implemented as separate component, however governed by the Export component.

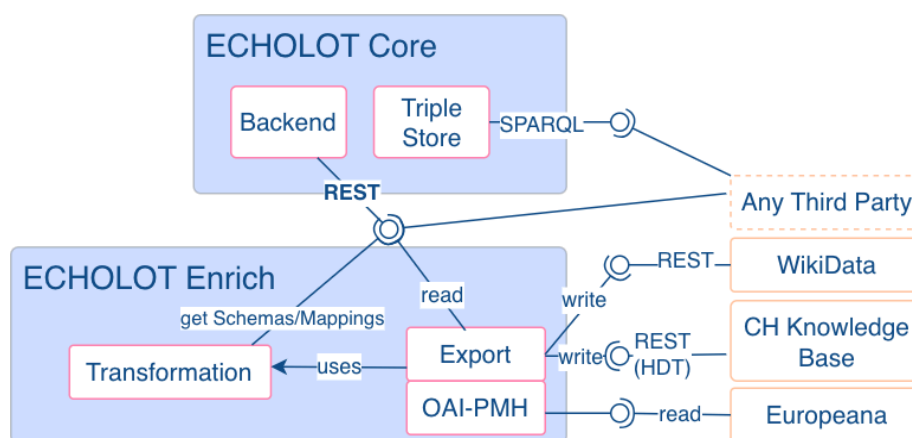


Figure 6. C3 – Export and Transformation component interactions

As a special case of the Import/Export Task, ECHOLOT aims to support “roundtripping”, i.e. bi-directional exchange of data between systems. The primary scenario is that data imported from source systems into ECHOLOT will be enriched and these enrichments should be made available in the source systems.

5.2.4 Reconciliation

The Reconciliation component implements the entity matching processes for data that has been imported into the ECHOLOT system. The entities will be reconciled against a set of knowledge bases, including the ECHOLOT data tier, and ones dictated by the primary target systems (CH Knowledge Base, Wikidata, Europeana). The component utilizes rule-, knowledge- and LLM-based methods: the traditional rule-based technique will be applied by default, and LLM-based processing for cases where the traditional method is not providing useful results.

The user should be able to select and specify the entities to be reconciled, the data sources to reconcile against and the used methods and their parameters. The component will use APIs of external knowledge bases (e.g., the [W3C Reconciliation Service API](#), when available). After the automatic reconciliation processing has run, the user will have the ability to check and validate the results. The system will record provenance information on the steps of the process (e.g., the used methods, their parameters and user inputs), by using the functionalities of the workflow engine and NeoWiki frontend as much as possible. The need for a specific **reconciliation cache** will be considered: its availability would make the reconciliation process faster by eliminating the need for continuous API calls to external services. On the negative side, the use of cache would make the reconciliation process more complex, by introducing problems related to cache invalidation, etc.

The following diagram depicts an early stage reconciliation prototype using LLM to improve the ranking of the results, developed by the UPV/EHU team as part of their work with test data from the WP5 Basque Heritage data:

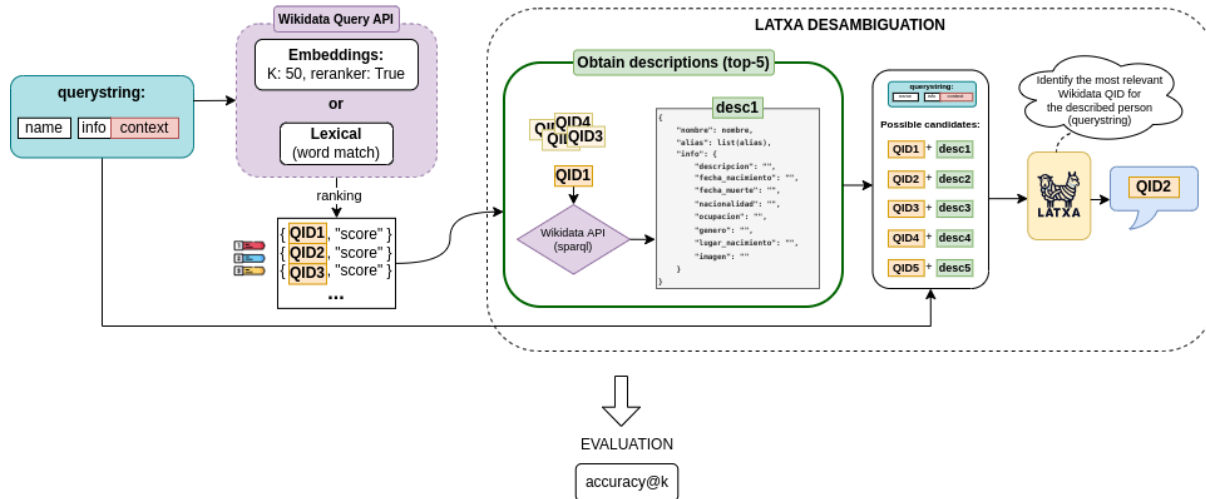


Figure 7. Reconciliation workflow prototype developed for testing Basque heritage data use cases

5.2.5 Enrichment

The enrichment component builds on the output of reconciliation. Once an entity has been matched to one or more external knowledge bases, ECHOLOT can use those links to augment the local record with additional data, and can apply generative methods to produce new descriptive content and to propose previously unknown relationships. Where Reconciliation answers the question “which known entity does this record correspond to?”, Enrichment answers “what else can we learn about this entity, and how can it be connected to the rest of the graph?”.

Two complementary modes of enrichment are foreseen. The first is retrieval-based enrichment, where the reconciled identifiers are used to pull additional statements, attributes and media from external sources (e.g., Wikidata, ECCCH KB and other ECHOLOT-supported systems) and to integrate them into the local record according to the ECHOLOT data model. The second is generative enrichment, where LLMs and other AI tools are used to generate new data, for example multilingual descriptions, summaries or suggested classifications, and to surface candidate links and hidden relationships that are not explicitly present in the source data. As in reconciliation, these methods are combined in a hybrid fashion and are always subject to human validation, in line with ECHOLOT’s human-in-the-loop philosophy.

Architecturally, enrichment follows the same pattern as reconciliation: a service exposing a unified interface through which it is invoked by the workflow manager, and which

orchestrates calls to a set of pluggable AI tools, agents and models. Because the two components share much of this orchestration logic, selection of inputs, dispatching of tasks to external or local tools, collection of results, and recording of provenance, the enrichment service may be deployed within the same service container as reconciliation rather than as a wholly separate unit. The distinguishing feature of enrichment is the range of AI back-ends it can route requests to.

To accommodate different deployment scenarios, institutional constraints and cost models, the component may support multiple model-hosting options behind a common abstraction:

- Support for a self-hosted, open-source model deployed alongside the ECHOLOT service. This may be a general-purpose open-weight LLM or a model fine-tuned for CH enrichment tasks and for the languages of the case studies, giving institutions full control over their data without reliance on third-party providers.
- A “bring your own API key” option, allowing institutions to connect commercial LLM providers using their own credentials. This lets users take advantage of state-of-the-art proprietary models where appropriate, while keeping billing and access governance on the institutional side and avoiding any central dependency on a single vendor within ECHOLOT itself.

Routing requests through a common abstraction means the enrichment workflows are decoupled from any particular model or provider, so that new tools and agents can be added as the technology landscape evolves and individual deployments can choose the option that best fits their privacy, sustainability and budget requirements.

6 Development Timeline

The project follows a 36-month timeline (January 2026 – December 2028), synchronized with the ECHOES infrastructure release schedule.

- **M1–M18 (Development Phase):** System design and release of initial open-source code. ECHOLOT’s **M18 (June 2027)** is synchronized with critical ECHOES milestones, including the release of the **Integration Manager** and **Security Management** components.
- **M12–M24 (Testing Phase):** In this phase, initial functioning prototypes of ECHOLOT’s Core and Enrich container functionalities are tested with WP5 case study partners

and results from the test provide new requirements to take into consideration during the final development stage.

- **M24-M36 (Iterative Improvement Phase):** After evaluation of the case study implementation results, further developments to the system are implemented, documented and integrated into ECCCH infrastructure. This phase also includes training and capacity building, among other sustainability and adoption measures.

7 References

Brown, S. (2026) *The C4 Model: Visualizing Software Architecture*. O'Reilly Media